Autonomous SAR Drone: Group Design Project Report

Team Members: Dmytro Chuprynyuk, Robin [Surname], [Member 3], [Member 4], [Member 5]

AENGM0074 Group Design Project — University of Bristol — March 2026

This report documents the group engineering process behind the design, development, and testing of an autonomous search and rescue (SAR) drone for the AENGM0074 Group Design Project at the University of Bristol. A team of five MSc students collaborated to build a quadcopter system capable of autonomous lawnmower search, onboard AI-based casualty detection using YOLOv8, operator-in-the-loop verification, and offset landing near a located target. We describe our project management approach including work breakdown structure, sprint-based scheduling, and role allocation. The systems engineering methodology — following a tailored V-model — is presented alongside requirements traceability, multi-criteria design trade studies, and a progressive test strategy that advanced from simulation through bench testing to field trials. Risk management, team communication via GitHub and regular meetings, and lessons learned are discussed. The project delivered a fully integrated system tested in simulation and on hardware, demonstrating effective interdisciplinary collaboration within a compressed academic timeline.

1 Introduction

1.1 Project Context

Search and rescue (SAR) operations in the United Kingdom face persistent challenges: large search areas, difficult terrain, limited personnel, and time-critical scenarios where every hour reduces the probability of finding a casualty alive [1, 2]. Unmanned aerial vehicles (UAVs) offer a compelling solution by providing rapid aerial coverage with sensor payloads that can detect casualties from altitude [3, 4].

This report documents the *group engineering process* behind the AENGM0074 Group Design Project at the University of Bristol, in which our team of five MSc students designed, built, and tested an autonomous SAR drone. While the companion technical report details the system's algorithms, hardware, and performance, this report focuses on *how* we worked as a team: our project management methodology, systems engineering approach, role allocation, risk management, and lessons learned.

1.2 Project Objectives

We defined the following top-level objectives at project inception:

1. **Autonomous search**: the drone takes off and flies a lawnmower pattern over a defined search polygon without manual piloting.
2. **AI-based detection**: an onboard YOLOv8 neural network running on a Raspberry Pi 5 detects a casualty (represented by a life-sized dummy) from the search altitude.
3. **Operator verification**: upon detection, the drone centres on the target and descends; a human operator confirms or rejects the detection via a ground station interface.
4. **Safe landing**: if confirmed, the drone lands at a 7.5 m offset from the casualty to avoid propwash injury, then marks the GPS location for ground teams.
5. **Progressive safety**: every capability is tested through a strict sequence — simulation, bench test (no propellers), passive flight (manual RC, AI observes only), and finally full autonomous flight — so that no untested behaviour is introduced to a live aircraft.

1.3 Team Composition

Our team comprised five members with complementary backgrounds spanning software engineering, flight dynamics, hardware integration, and project management (Table 1).

Table 1: Team composition and primary responsibilities.

Member	Role	Focus Area
Dima	CV / Software Lead	Computer vision, AI detection, Pi software, simulator, ground station
Robin	Flight Dynamics	State machine integration, flight parameter tuning, main.py testing Member 3¿Member 3
	Pilot	RC flying, kill switch operation, manual override procedures Member 4¿Member 4
	Hardware Lead	Frame assembly, wiring, Cube flight controller setup, power systems Member 5¿Member 5
	PM / Report Lead	Project management, scheduling, risk assessment, report coordination

1.4 Report Structure

The remainder of this report is organised as follows. Section 2 describes our work breakdown structure, timeline, and sprint methodology. Section 3 presents the systems engineering framework including the V-model, requirements, and trade studies. Section 4 details role allocation, the RACI matrix, and communication plan. Section 5 covers the risk register and mitigation strategies. Section 6 describes our progressive testing methodology. Section 7 reflects on what worked, what did not, and recommendations for future teams. Section 8 summarises the group's achievements.

2 Project Management

2.1 Management Philosophy

We adopted a hybrid project management approach combining traditional plan-driven methods with agile practices, a strategy well-suited to hardware-software projects with fixed deadlines [5, 6]. The academic calendar imposed a hard deadline, making a purely agile approach risky; however, the exploratory nature of integrating AI with flight hardware demanded iterative development cycles. Our solution was to use a traditional work breakdown structure (WBS) and milestone plan for high-level scheduling, while executing work in informal two-week sprints with regular check-ins.

2.2 Work Breakdown Structure

We decomposed the project into five parallel workstreams, each with clearly defined deliverables and integration points:

Table 3: Work breakdown structure – top-level workstreams.

Stream	Owner	Scope	Hardware?
A: Computer Vision	Dima	Camera, AI model, TFLite inference on Pi	Pi + Camera
B: Hardware & Connectivity	[Member 4]	Wiring, serial link, GPS, RC binding	All hardware
C: Search Logic	Robin	Lawnmower planning, state machine, algorithms	No (simulation)
D: Ground Station	Dima / Robin	Mission Planner, SSH, web dashboard, operator UI	Telemetry radio
E: Safety & Testing	All	Preflight checks, failsafes, progressive test plan	All hardware

Each workstream was further divided into numbered tasks (e.g. A1: camera works on Pi, A2: AI detection works on Pi, etc.), producing approximately 25 discrete work packages. Streams A and B required physical hardware and could only begin once components arrived; Streams C and D could proceed immediately using laptop simulation and SITL (Software-In-The-Loop) [7].

2.3 Timeline and Milestones

The project spanned approximately 12 weeks from initial planning to final demonstration. Table 5 summarises the key milestones.

Table 5: Project milestones and target dates.

ID	Week	Milestone	Status
M1	1–2	Requirements defined, WBS agreed, roles assigned	Complete
M2	2–3	Simulation working end-to-end (state machine + CV + search pattern)	Complete
M3	4–5	Pi hardware verified: camera, TFLite inference, Cube serial link	Complete
M4	5–6	Integration 1: Pi runs detection + Cube telemetry simultaneously	Complete
M5	6–7	Ground station operational (web dashboard, Mission Planner connected)	Complete
M6	7–8	Integration 2: full bench test with real camera + simulated flight	Complete
M7	8–9	Model retrained on real flight video data (v2, mAP50 = 0.995)	Complete
M8	9–10	Field day: bench tests, FOV calibration, passive flight observation	Complete
M9	11–12	Final demonstration and report submission	Complete

1. **Sprint planning** (group meeting): review progress against milestones, assign tasks for the next two weeks, identify blockers.
2. **Execution**: individual or pair work on assigned tasks. Code changes pushed to feature branches on GitHub.
3. **Integration checkpoint**: at the end of each sprint, we merged completed work into the main branch and ran integration tests.
4. **Retrospective** (brief, often informal): what went well, what to improve, any new risks.

We did not use formal story points or velocity tracking; the small team size (five members) and direct communication made heavyweight agile tooling unnecessary. Instead, we maintained a shared status document (`GROUP_STATUS.md`) in the repository that tracked each member's current tasks, blockers, and completed work.

2.5 Tool Chain

Table 7: Tools used for project management and development.

Tool	Purpose
GitHub	Version control, branch management, code review, issue tracking
Git branches	Feature isolation (`MainOne2`–`MainOne8`, `Working*` branches)
Group chat	Day-to-day communication, quick decisions
Weekly meetings	Sprint planning, integration checkpoints, design reviews
Mission Planner	Flight controller configuration, telemetry monitoring, geofence setup
Google Colab	GPU-accelerated model training (YOLOv8 retraining)
Overleaf / LaTeX	Collaborative report writing
Project dashboard	Custom React web app for WBS tracking and SE visualisation

2.6 Scheduling Challenges

The primary scheduling challenge was the dependency between software development and hardware availability. Streams A and B could not begin until the Raspberry Pi, camera module, and Cube flight controller were physically available. We mitigated this by:

- **Simulation-first development**: the entire state machine, search pattern generator, and detection pipeline were developed and tested in simulation on laptops before any hardware arrived. When the Pi became available, the same code ran with minimal configuration changes.
- **Dual-backend vision system**: `vision.py` was designed with automatic backend detection — Ultralytics (PyTorch) on laptops, TFLite on the Pi — so that CV development could proceed on any platform.
- **Docker-based Pi simulation**: a Dockerfile replicated the Pi's lightweight dependency stack, allowing us to verify that TFLite models loaded correctly on an ARM-like environment before physical Pi testing.

Weather also constrained field testing. Our first scheduled flight day was cancelled due to high winds, reducing the available outdoor test time and forcing us to maximise the value of bench testing and indoor validation.

3 Systems Engineering

3.1 V-Model Framework

We adopted a tailored V-model [9] as our systems engineering lifecycle, mapping the left side (decomposition and design) to our early simulation-based development and the right side (integration and verification) to our progressive hardware testing. The V-model is well-suited to safety-critical systems where requirements traceability and staged verification are essential [10, 11].

The iterative nature of our sprint methodology meant we traversed the V-model multiple times at increasing fidelity: first in pure simulation, then with hardware-in-the-loop (camera + SITL), and finally with the complete physical system.

Table 9: V-model mapping for the SAR drone project.

Left side	Activity	Right side	Verification
System requirements	Mission objectives, constraints, regulations	System validation	Full autonomous flight test
Subsystem design	CV, autopilot, comms, ground station	Integration testing	Bench test (all subsystems, no props)
Component design	TFLite model, state machine, planning algorithm	Component testing	Unit tests, simulation, benchmarks
Implementation	Python code, hardware assembly	—	—

Table 11: System-level requirements.

ID	Requirement	Verification
SR-01	The drone shall autonomously search a user-defined polygon at a configurable altitude	Simulation + flight test
SR-02	The onboard AI shall detect a casualty-sized target with >90% recall from the search altitude	CV benchmark + field test
SR-03	A human operator shall confirm or reject each detection before the drone takes action	Ground station UI test
SR-04	The drone shall land at a safe offset ($\geq$5 m) from a confirmed target	Simulation + GPS accuracy test
SR-05	An RC pilot shall be able to override autonomous control at any time via kill switch	RC override test
SR-06	The system shall comply with UK CAA regulations for the specific operational category [12, 13]	Documentation review
SR-07	The system shall respect no-fly zone boundaries (SSSI geofence)	Simulation + parameter check
SR-08	All autonomous capabilities shall be validated through progressive testing before live flight	Test plan review

3.2 Requirements

We defined requirements at three levels: system, subsystem, and component. Table 11 presents the key system-level requirements.

Requirements were traced forward to design decisions, test cases, and verification evidence. For example, SR-02 drove the selection of YOLOv8n as the detection model (balancing accuracy and inference speed on the Pi), while SR-05 drove the implementation of RC override at every state in the finite state machine.

3.3 Design Trade Studies

We conducted multi-criteria decision analysis (MCDA) for key design choices, following Pugh's method [14] with weighted scoring. We document three representative trade studies below.

3.3.1 Detection Model Selection

We evaluated three model architectures for onboard casualty detection:

Table 13: Detection model trade study (1–5 scale, 5 = best).

Criterion	Weight	YOLOv8n	YOLOv8s	SSD MobileNet	Faster R-CNN
Inference speed on Pi	0.30	4	3	5	1
Detection accuracy (mAP)	0.30	4	5	3	5
Model size (TFLite)	0.15	5	3	5	1
Ease of retraining	0.15	5	5	3	3
Community/documentation	0.10	5	5	4	4
Weighted score		**4.35**	3.95	3.95	2.50

YOLOv8n was selected for the best balance of speed and accuracy. Its TFLite export ran at 4.8 FPS on the Pi 5 (206 ms per frame), which was sufficient for the drone's search speed of 3–5 m/s.

3.3.2 Video Streaming Protocol

For live camera feed from the Pi to the ground station, we evaluated six streaming approaches:

Table 14: Video streaming trade study.

Criterion	Weight	MJPEG/HTTP	H.264/GStreamer	WebRTC
Reliability in field	0.35	5	3	3
Browser compatibility	0.25	5	2	5
Dependencies required	0.20	5	2	1
Latency	0.10	3	5	5
Bandwidth efficiency	0.10	2	5	5
Weighted score		**4.55**	2.85	3.30

MJPEG was chosen because reliability and simplicity outweighed latency for a passive monitoring application at 3–5 FPS. The stateless HTTP protocol meant any browser could connect without special codecs or software.

3.3.3 Companion Computer Selection

Table 15: Companion computer trade study.

Criterion	Weight	Raspberry Pi 5	Jetson Nano	Intel NUC
AI inference speed	0.25	3	5	4
Weight / size	0.25	5	4	1
Power consumption	0.20	5	3	1
Cost	0.15	5	3	2
Ecosystem / support	0.15	5	4	3
Weighted score		**4.50**	3.90	2.05

The Raspberry Pi 5 [15] was selected for its excellent power-to-weight ratio, extensive ecosystem, and adequate CPU performance for TFLite inference. The Jetson Nano offered better GPU acceleration but at higher power draw and cost.

3.4 Configuration Management

All source code, configuration files, and documentation were managed in a single GitHub repository. We followed a branching strategy with:

- A **main branch** (`MainOne2` through `MainOne8`) for stable, tested code.
- **Feature branches** (`Working*`) for active development, merged only after testing.
- **Tagged commits** at each milestone for rollback capability.

Configuration parameters (altitudes, speeds, thresholds, camera settings) were centralised in a single `config.py` file with automatic hardware detection, ensuring the same codebase ran on both laptops (simulation) and the Pi (real hardware) without code changes [16].

4 Team Roles and Communication

4.1 Role Allocation

Roles were assigned based on individual expertise, interest, and prior experience. We aimed for each member to have a clearly defined primary responsibility while maintaining enough cross-training that no single absence would block progress [17]. The team naturally formed around a T-shaped competency model: each member had deep expertise in one area (the vertical bar of the T) and broad understanding of the overall system (the horizontal bar).

4.2 RACI Matrix

We used a RACI (Responsible, Accountable, Consulted, Informed) matrix to clarify ownership of key deliverables and avoid ambiguity [5]. Table 16 presents the matrix for major project activities.

4.3 Communication Plan

Effective communication was critical given the interdisciplinary nature of the project and the need to coordinate hardware availability, software integration, and field testing logistics. We established the following communication channels:

4.3.1 Decision Documentation

All significant technical decisions were recorded in a `DESIGN_DECISIONS.md` file using a structured format: context, options considered, decision, rationale, and trade-offs accepted. This served two purposes: (1) it prevented re-litigating

Table 16: RACI matrix for key project activities. R = Responsible, A = Accountable, C = Consulted, I = Informed.

Activity	Dima	Robin	Pilot	Hardware	PM
CV model development	R/A	I	I	I	I
State machine design	C	R/A	I	I	I
Pi software integration	R/A	C	I	C	I
Hardware assembly & wiring	C	C	I	R/A	I
Flight controller config	I	C	C	R/A	I
RC setup & kill switch	I	I	R/A	C	I
Flight test piloting	I	C	R/A	I	I
Ground station operation	C	R/A	I	I	I
Risk assessment	C	C	C	C	R/A
Report writing	C	C	C	C	R/A
Schedule management	I	I	I	I	R/A
Simulation development	R/A	C	I	I	I
Progressive test planning	R/A	C	C	C	C
Search area survey	I	I	I	I	R/A

Table 17: Communication channels and their purposes.

Channel	Frequency	Purpose
Group chat	Daily	Quick questions, sharing results, coordination
Weekly meeting	Weekly	Sprint planning, progress review, design decisions
GitHub pull requests	As needed	Code review, integration approval
`GROUP_STATUS.md`	Continuous	Persistent record of decisions, blockers, action items
Ad-hoc pair sessions	As needed	Debugging hardware issues, integration testing

4.4 Team Dynamics

The team progressed through Tuckman's stages of group development [18] in an accelerated fashion due to the short project timeline:

- **Forming** (weeks 1–2): initial meetings, getting to know each other's skills, agreeing on tools and workflow.
- **Storming** (weeks 2–3): negotiating responsibilities, resolving differences in coding style and development workflow (e.g. branch naming, commit conventions).
- **Norming** (weeks 3–5): established routines, clear ownership, effective use of GitHub and group chat.
- **Performing** (weeks 5–12): focused execution, parallel workstreams running smoothly, integration checkpoints well-attended.

A key enabler of smooth collaboration was the **modular software architecture**. Because each major component (`vision.py`, `planning.py`, `config.py`, `main.py`) had well-defined interfaces, team members could work on their respective areas without creating merge conflicts or breaking each other's code. The vision system, for example, exposed a single function — `detect_in_image(frame)` returning `(found, x, y, conf)` — that all other modules consumed. This interface contract meant the CV developer could change model backends, preprocessing, or inference logic without touching any other file.

4.5 Knowledge Transfer

Given the specialised nature of certain components (particularly the AI pipeline and MAVLink protocol), we invested in knowledge transfer activities:

- **Documentation**: comprehensive `CLAUDE.md` (project ground truth), architecture documents, setup guides for each platform, and flight day checklists — all version-controlled alongside the code.
- **Pair programming**: critical integration points (e.g. connecting the Pi to the Cube flight controller via MAVProxy) were done in pairs to spread knowledge.
- **Written guides**: step-by-step setup documents (`PI_SETUP.md`, `FLIGHT_DAY_CHECKLIST.md`) ensured any team member could reproduce the environment independently.

- **Interactive simulator**: `simple_simulator.py` allowed any team member to experience the full mission flow on their laptop without hardware, building shared understanding of the state machine and detection logic.

5 Risk Management

5.1 Approach

We adopted a structured risk management process aligned with ISO 31000 principles [19], using a risk register that was reviewed at each sprint planning meeting. Risks were assessed on a 5×5 likelihood–impact matrix, with mitigation strategies defined for all risks rated medium or above [20].

5.2 Risk Register

Table 19 presents the key risks identified throughout the project, their initial ratings, mitigations applied, and residual ratings.

Table 19: Risk register. L = Likelihood (1–5), I = Impact (1–5), R = Risk score (L×I).

ID	Risk	L	I	R	Mitigation	Residual
R1	AI fails to detect dummy from search altitude	3	5	15	Retrained model on real aerial data; adjustable altitude and confidence threshold	6
R2	Pi inference too slow for flight speed	3	4	12	Benchmarked at 4.8 FPS; search speed reduced to match; threaded pipeline option	4
R3	GPS estimation error exceeds 5 m	3	3	9	Inverse variance weighting, multi-pass averaging, Kalman filter; 7.5 m offset landing for margin	4
R4	Weather cancels flight testing	4	4	16	Scheduled backup dates; maximised bench testing value; indoor validation of all software	8
R5	Hardware damage during testing	2	5	10	Progressive test plan (no skipping steps); RC kill switch always active; tethered hover before free flight	4
R6	Team member unavailable	3	3	9	Cross-training via pair programming; comprehensive documentation; modular code with clear interfaces	4
R7	Serial communication failure (Pi–Cube)	3	4	12	MAVProxy UDP bridge bypasses Python 3.13 serial bug; tested early in project	3
R8	Drone enters SSSI no-fly zone	2	5	10	Software geofence with 20 m buffer; speed-cap near boundary; auto-switch to manual if breached	3
R9	Camera colour calibration incorrect	3	3	9	Empirically tested all 6 RGB permutations; documented BGR finding; lens calibration applied	2
R10	Scope creep delays core deliverables	3	3	9	Fixed core feature set early; deferred enhancements to `NICE_TO_HAVE.md` backlog	4

5.3 Risk Mitigation Strategies

Several cross-cutting strategies addressed multiple risks simultaneously:

5.3.1 Simulation-First Development

The most effective risk mitigation was developing and testing the entire system in simulation before touching real hardware. By the time we connected the Pi to the Cube flight controller, the state machine had already been validated through hundreds of simulated missions. This approach — consistent with NASA's progressive V&V methodology for UAS [21] — meant that hardware testing focused on *integration* rather than debugging logic errors.

5.3.2 Progressive Test Escalation

We defined five test levels, each introducing exactly one new variable (Section 6). This strategy directly mitigated R5 (hardware damage) by ensuring untested code never controlled a live aircraft, and R1 (detection failure) by calibrating CV performance in controlled conditions before flight.

5.3.3 Multiple Model Variants

To mitigate R1, we maintained three trained model variants (`cv_models/sar_v2_1088`, `sar_640`, `sar_1280`) plus a generic COCO person detector as a fallback. All models were drop-in replacements with identical input/output shapes. On flight day, swapping models required a single file copy and script restart — no code changes.

5.3.4 Dual Communication Path

To mitigate R7, the MAVProxy bridge provided a battle-tested serial-to-UDP gateway, simultaneously solving the Python 3.13 serial compatibility issue and enabling Mission Planner connectivity via TCP. This architectural decision (DD-04) turned a blocking risk into a feature.

5.4 Risk Events That Materialised

Two risks materialised during the project:

1. **R4 (Weather)**: Our first scheduled flight day was cancelled due to high winds. We used the field day for bench testing, FOV calibration, and lens calibration instead, extracting substantial value from the trip despite not flying.
2. **R9 (Camera colour)**: The IMX296 sensor output BGR data despite being labelled RGB888 by picamera2. We discovered this empirically by testing all six channel permutations. The fix (removing an unnecessary `cvtColor` call) was trivial once diagnosed, but would have caused detection failures if not caught during bench testing.

Both events were handled without schedule impact because our mitigations were already in place.

5.5 Regulatory Compliance

Operating a UAV in UK airspace required compliance with UK CAA regulations [12, 13] and the JARUS SORA framework [22]. Key compliance measures included:

- Flight within the university's approved operational area at Fenswood Farm.
- Qualified pilot with appropriate CAA registration.
- Risk assessment submitted and approved before each flight session.
- SSSI no-fly zone respected via software geofence with graduated speed reduction.
- RC override available at all times; operator-in-the-loop for all landing decisions.

6 Testing and Validation

6.1 Testing Philosophy

Our testing approach was guided by one principle: *never skip a step.* Each test level introduces exactly one new variable, so that if a test fails, the cause is immediately identifiable as the newly introduced element [23, 24]. This progressive escalation mirrors NASA's UAS verification methodology [21] and is formalised in design decision DD-07.

6.2 Five-Level Test Progression

6.3 Test Script Organisation

We developed over 40 test scripts, organised into purpose-driven directories:

The flight test scripts were explicitly numbered (0a through 4) to enforce ordering. Each script's header documented prerequisites, what it tests, and how to interpret results.

6.4 Integration Testing

We defined four formal integration checkpoints, each requiring all prerequisite workstream tasks to pass before proceeding:

1. **Integration 1 — Pi Works**: Camera detection + Cube telemetry working simultaneously on the Pi (workstreams A1–A4 + B1).

Table 21: Progressive test levels. Each level builds on the previous.

Level	Name	What It Proves	Risk Introduced
0	Mission Planner AUTO	Cube, GPS, motors, RTL all work. No custom code involved.	Propellers spinning
1	Waypoint test script	Our MAVLink commands (arm, takeoff, goto, land) work on real hardware. No camera.	Custom flight commands
2	Manual flight + passive CV	CV detects dummy from altitude in real outdoor conditions. Pi sends zero commands.	CV in real environment
3	Autonomous search, log only	Full search pattern autonomous, CV detects and logs but never triggers action.	Autonomous navigation
4	Full autonomous mission	Everything enabled: search, detect, centre, descend, verify, land.	Autonomous decision-making

Table 23: Test script categories and their purposes.

Directory	Question Answered	Examples
`tests/hardware/`	Does this component work?	`benchmark.py`, `gps_test.py`, `buzzer_test.py`
`tests/flight/`	Can it fly?	`0a_cube_commands.py` through `4_detect_and_center.py`
`tests/diagnostics/`	Is it working now?	`diagnostics.py`, `cube_monitor.py`, `camera_stream.py`
`tests/calibration/`	Are the numbers right?	`fov_calibrate.py`, `lens_calibrate.py`, `gps_ground_truth.py`
`tests/day_1_experiments/`	What does the data show?	`altitude_sweep.py`, `speed_sweep.py`, `gps_accuracy.py`
`tests/laptop/`	Development tools	`video_test.py`, `test_tflite.py`, `debug_tflite.py`

2. **Integration 2 — Full System on Bench**: Real camera + simulated flight via SITL + ground station connected (workstreams A + B + D).
3. **Integration 3 — Ground Test**: Full state machine on real Cube hardware, no propellers, all failsafes verified (workstreams A + B + C + E).
4. **Integration 4 — Flight**: Complete autonomous mission with propellers (all workstreams).

6.5 Verification and Validation (V&V)

6.5.1 Verification

Verification — confirming the system was built correctly — was performed through:

- **Simulation testing**: hundreds of end-to-end missions in SITL, validating state transitions, waypoint following, detection triggering, and landing logic.
- **Code review**: pull requests reviewed by at least one other team member before merging to the main branch.
- **Benchmarking**: quantitative performance measurement (inference speed, detection rate, GPS estimation accuracy) against defined thresholds.
- **Preflight checker**: automated script (`preflight.py`) that verified camera, AI model, and Cube connectivity before every test session.

6.5.2 Validation

Validation — confirming we built the right system — was performed through:

- **Demonstration to supervisors**: showing the simulator and bench test results confirmed alignment with project objectives.
- **Video analysis**: replaying DJI flight footage through our detection pipeline validated that the CV system would perform in realistic conditions (`tests/laptop/video_test.py`).
- **Field testing**: bench tests at Fenswood Farm with the complete hardware stack validated real-world performance of FOV calculations, lens calibration, and detection from altitude.

6.6 Key Test Results

Table 25: Summary of key test results.

Test	Result	Requirement
TFLite inference speed (Pi 5)	206.5 ms / 4.8 FPS	<200 ms (marginal)
Detection rate (benchmark, 50 runs)	50/50 (100%)	>90%
Detection confidence (benchmark)	0.966 mean	>0.4 threshold
Lens undistortion overhead	+1.5 ms	Negligible
Retrained model mAP50	0.995	>0.9
FOV calibration accuracy	1.7–1.9 m measured vs 1.8 m actual	<10% error
GPS estimation (simulation)	∼0.5 m error after centring	<5 m
GPS estimation (DJI video replay)	CEP50 = 2.3 m	<5 m
State machine (simulation)	All 11 states + 32 transitions verified	Complete coverage

6.7 Experiment Design for Flight Day

We designed structured experiments for flight testing, each with predefined data collection protocols and CSV output:

- **Altitude sweep**: detection rate vs altitude in 5 m increments (10–35 m) to determine maximum reliable detection height.
- **Speed sweep**: detection rate vs drone speed with blur metric to determine maximum search speed.
- **GPS accuracy**: estimated target position vs known GPS coordinates to validate the projection mathematics.
- **GPS drift**: stationary noise measurement (CEP50, CEP95) to characterise baseline GPS uncertainty.
- **Model comparison**: side-by-side benchmark of all available TFLite models under identical conditions.

These experiments were implemented as standalone Python scripts in `tests/day_1_experiments/`, each producing CSV files for post-flight analysis. All scripts were passive observers that sent zero commands to the flight controller, ensuring data collection could not endanger the aircraft.

7 Lessons Learned

7.1 What Worked Well

7.1.1 Simulation-First Development

The decision to build and test the entire system in simulation before any hardware was available was the single most impactful engineering choice we made. By the time the Raspberry Pi and flight controller arrived, we had a working state machine, search pattern generator, detection pipeline, and ground station — all validated through hundreds of simulated missions. Hardware integration became a matter of configuration rather than debugging. We strongly recommend this approach for any team working on autonomous systems with limited hardware access.

7.1.2 Modular Architecture with Clean Interfaces

Isolating the vision system behind a single function (`detect_in_image(frame) -> (found, x, y, conf)`) meant that five people could work on the project simultaneously without merge conflicts. The CV developer changed models, preprocessing, and backends multiple times without any other team member needing to update their code. The dual-backend system (Ultralytics on laptops, TFLite on Pi) was particularly effective: it let us develop and test on any platform and deploy to the Pi with zero code changes.

7.1.3 Progressive Test Strategy

The numbered test progression (0a through 4) prevented us from making the common mistake of attempting a full autonomous flight before validating individual components. When issues arose (e.g. the camera colour bug), they were caught in controlled bench tests rather than mid-flight. The strict "never skip a step" rule, while sometimes frustrating when progress felt slow, saved us from potential hardware damage.

7.1.4 Decision Documentation

Recording design decisions in a structured format (DD-01 through DD-10) proved valuable beyond the immediate project. When writing the report, we had ready-made justifications with context, alternatives considered, and rationale. When new team members had questions about "why do we do it this way?", we could point them to the relevant DD entry rather than reconstructing the reasoning from memory.

7.1.5 Comprehensive Test Script Library

The 40+ test scripts, organised by category and purpose, served as both verification tools and documentation. Each script answered a specific question ("does the GPS work?", "how fast is inference?", "can we detect from 25 m?") and produced quantitative output. This library was particularly valuable on field days, where limited time made it essential to run the right test efficiently.

7.2 What Could Be Improved

7.2.1 Earlier Hardware Integration

While simulation-first was the right strategy, we could have integrated hardware earlier in the project. The Pi, camera, and Cube were available for several weeks before we assembled and tested them together. Earlier integration would have given us more time to discover and resolve issues like the camera colour bug and the Python 3.13 serial compatibility problem.

7.2.2 More Structured Sprint Ceremonies

Our informal sprint methodology worked adequately for a five-person team, but we occasionally lost track of individual progress between meetings. A more disciplined approach — brief daily standups, even via text — would have helped identify blockers sooner. The `GROUP_STATUS.md` document was useful but only effective when team members remembered to update it.

7.2.3 Cross-Training Depth

Despite our knowledge transfer efforts, certain critical capabilities remained concentrated in one person. The AI pipeline and MAVLink protocol were primarily understood by one team member, creating a bus factor of one for those subsystems. More structured pair programming sessions earlier in the project would have distributed this knowledge more effectively.

7.2.4 Flight Testing Time

Weather cancellation reduced our available outdoor testing to a single field day, which was used primarily for bench testing and calibration rather than flight. In hindsight, we should have booked multiple flight slots early in the project to create scheduling resilience. We also underestimated the logistical overhead of field testing (transport, setup, battery management, weather monitoring).

7.2.5 Scope Management

The project accumulated a substantial feature backlog (16 items in `NICE_TO_HAVE.md`) that sometimes distracted from core deliverables. While having a structured backlog prevented uncontrolled scope creep, we should have been more disciplined about deferring non-essential features earlier. For example, the NFZ geofence with three avoidance strategies (DD-10) was an impressive engineering exercise but consumed development time that could have been spent on additional flight testing.

7.3 Recommendations for Future Teams

Based on our experience, we offer the following recommendations for future groups undertaking similar autonomous systems projects:

1. **Start with simulation, but integrate hardware as soon as it arrives.** Do not wait for perfect software before testing on real hardware — the bugs you find in integration are different from the bugs you find in simulation.
2. **Book multiple flight/test slots early.** Weather, equipment failures, and scheduling conflicts will cancel at least one session. Plan for it.

3. **Document decisions as you make them.** A structured decision log is far more useful than meeting minutes. Future you (and your report) will thank present you.
4. **Define interfaces before dividing work.** Agree on function signatures, data formats, and communication protocols at the start. This enables truly parallel development.
5. **Build a test script for every component.** Individual, purpose-built test scripts are more valuable than a single "test everything" script. They are faster to run, easier to debug, and serve as living documentation.
6. **Use progressive testing religiously.** The temptation to skip steps is strong when deadlines loom. Resist it. A crashed drone sets the project back weeks; a methodical test progression costs hours.
7. **Version control everything.** Code, configuration, documentation, design decisions, meeting notes — all in the same repository. Context switching between tools loses information.
8. **Invest in developer experience.** Time spent making the codebase easy to work with (clear file structure, auto-detection of hardware, comprehensive `--help` flags, readable error messages) pays dividends throughout the project.

8 Conclusion

This report has documented the group engineering process behind the AENGM0074 autonomous SAR drone project. A team of five MSc students collaborated over 12 weeks to design, build, and test a quadcopter system capable of autonomous search, AI-based casualty detection, operator-in-the-loop verification, and offset landing.

Our key achievements as a team were:

- **Effective parallel development**: five workstreams running simultaneously with clear interfaces and integration checkpoints, enabled by a modular software architecture.
- **Simulation-first validation**: the complete system was validated in simulation before hardware integration, dramatically reducing integration risk and debugging time.
- **Progressive test methodology**: a five-level test progression ensured that no untested capability was introduced to a live aircraft, with over 40 purpose-built test scripts providing comprehensive coverage.
- **Structured decision-making**: multi-criteria trade studies for key design choices (detection model, streaming protocol, companion computer) documented in a persistent design decisions log.
- **Proactive risk management**: a risk register reviewed at each sprint, with cross-cutting mitigations (simulation-first, multiple model variants, dual communication paths) that addressed multiple risks simultaneously.
- **Comprehensive documentation**: architecture documents, setup guides, flight checklists, and blueprints version-controlled alongside the code, ensuring knowledge persistence across the team.

The project also highlighted areas for improvement, particularly around earlier hardware integration, more structured sprint ceremonies, deeper cross-training, and booking sufficient outdoor testing time. These lessons are offered to future teams undertaking similar projects.

Despite the constraints of a compressed academic timeline and weather-limited field access, the team delivered a fully integrated system that was validated through bench testing, video analysis of real flight footage, and calibrated hardware measurements. The autonomous SAR drone project demonstrated that effective interdisciplinary collaboration, structured systems engineering, and disciplined progressive testing can deliver a complex autonomous system within the tight constraints of a university group project.

References

[1] R. R. Murphy. *Disaster Robotics*. Cambridge, MA: MIT Press, 2014.

[2] M. A. Goodrich et al. "Supporting Wilderness Search and Rescue Using a Camera-Equipped Mini UAV". In: *Journal of Field Robotics* 25.1–2 (2008), pp. 89–110.

[3] M. Silvagni et al. "Multipurpose UAV for Search and Rescue Operations in Mountain Avalanche Events". In: *Geomatics, Natural Hazards and Risk* 8.1 (2017), pp. 18–33.

[4] B. Mishra et al. "Drone-surveillance for Search and Rescue in Natural Disaster". In: *Computer Communications* 156 (2020), pp. 1–10.

[5] Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 7th. Newtown Square, PA: Project Management Institute, 2021.

[6] B. W. Boehm. "A Spiral Model of Software Development and Enhancement". In: *Computer* 21.5 (1988), pp. 61–72.

[7] ArduPilot Dev Team. *ArduPilot SITL Simulator: Software in the Loop Simulation*. https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html. Accessed: 2026. 2024.

[8] K. Schwaber. "SCRUM Development Process". In: *Business Object Design and Implementation*. Springer, 1997, pp. 117–134.

[9] K. Forsberg and H. Mooz. "The Relationship of System Engineering to the Project Cycle". In: *Engineering Management Journal* 3.3 (1991), pp. 36–43.

[10] INCOSE. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. 4th. John Wiley & Sons, 2015.

[11] B. S. Blanchard and W. J. Fabrycky. *Systems Engineering and Analysis*. 5th. Upper Saddle River, NJ: Prentice Hall, 2011.

[12] UK Civil Aviation Authority. *The Drone and Model Aircraft Code*. https://www.caa.co.uk/drones/. Accessed: 2026. 2023.

[13] UK Civil Aviation Authority. *CAP 722: Unmanned Aircraft System Operations in UK Airspace*. https://publicapps.caa.co.uk/. Accessed: 2026. 2024.

[14] S. Pugh. *Total Design: Integrated Methods for Successful Product Engineering*. Wokingham, UK: Addison-Wesley, 1991.

[15] Raspberry Pi Foundation. *Raspberry Pi 5 Documentation*. https://www.raspberrypi.com/documentation/computers/raspberry-pi-5.html. Accessed: 2026. 2023.

[16] S. Chacon and B. Straub. *Pro Git*. 2nd. Available at https://git-scm.com/book/en/v2. Apress, 2014.

[17] J. R. Katzenbach and D. K. Smith. *The Wisdom of Teams: Creating the High-Performance Organization*. Harvard Business Review Press, 2015.

[18] B. W. Tuckman. "Developmental Sequence in Small Groups". In: *Psychological Bulletin* 63.6 (1965), pp. 384–399.

[19] International Organization for Standardization. *ISO 31000:2018 – Risk Management Guidelines*. International Standard. ISO, 2018.

[20] D. Hillson and P. Simon. *Practical Project Risk Management: The ATOM Methodology*. 2nd. Vienna, VA: Management Concepts, 2012.

[21] National Aeronautics and Space Administration. *Unmanned Aircraft Systems (UAS) Integration in the National Airspace System (NAS) Project: Test and Evaluation*. Tech. rep. NASA/TM-2015-218817. Describes progressive V&V methodology for UAS. NASA, 2015.

[22] JARUS. *JARUS Guidelines on Specific Operations Risk Assessment (SORA)*. Tech. rep. Edition 2.0. JAR-DEL-WG6-D.04. Joint Authorities for Rulemaking on Unmanned Systems, 2019.

[23] P. Koopman and M. Wagner. "Challenges in Autonomous Vehicle Testing and Validation". In: *SAE International Journal of Transportation Safety*. Vol. 4. 1. 2016, pp. 15–24.

[24] G. J. Myers, C. Sandler, and T. Badgett. *The Art of Software Testing*. 3rd. John Wiley & Sons, 2011.